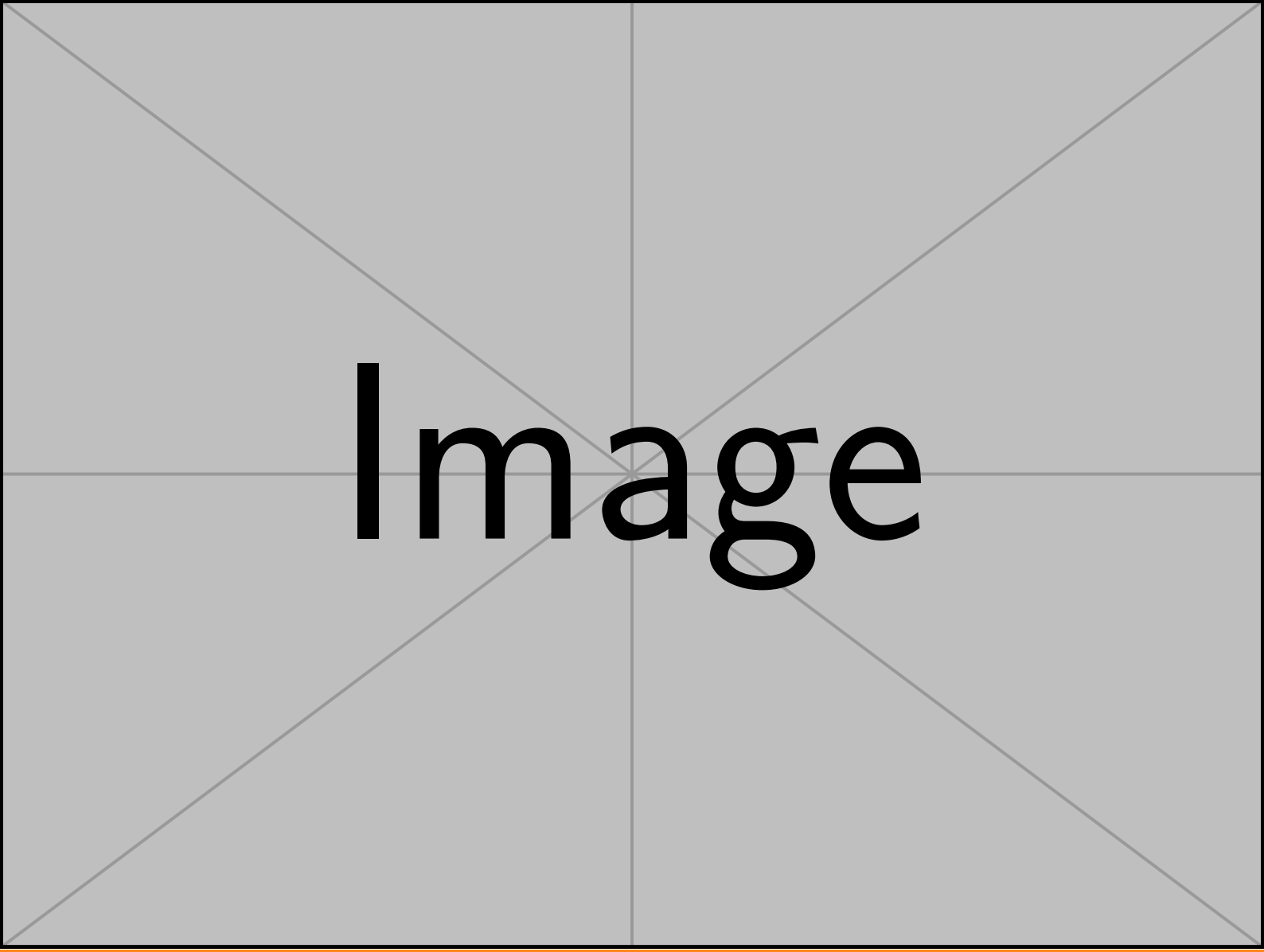


Slip User Manual

A Minimal HDL DSL Compiler

Author: Slip Project

Date: October 15, 2026

Version: 1.1

Contents

Chapter 1 Introduction **1**
1.1 Design Philosophy 1

Chapter 2 Installation **2**
2.1 Prerequisites 2
2.2 Build and Install 2

Chapter 3 Command Line Interface **3**
3.1 `slip build` — Compile to SystemVerilog 3
3.2 `slip check` — Validate without generating output 3
3.3 Project Configuration 3
3.4 Inspection and Machine-Readable Diagnostics 4

Chapter 4 Language Overview **5**
4.1 Key Differences from SystemVerilog 5

Chapter 5 Module Declaration **6**
5.1 Basic Syntax 6
5.2 With Parameters 6
5.3 Implicit Ports 6

Chapter 6 Parameters and Localparams **7**
6.1 Parameters (`param`) 7
6.2 Localparams (`localparam`) 8

Chapter 7 Signals **10**
7.1 Explicit Declaration 10
7.2 Implicit Declaration 10

Chapter 8 Port Inference **11**
8.1 Example 11

Chapter 9 Assignments **12**
9.1 Continuous Assignment 12
9.2 Procedural Assignment 12

Chapter 10 Sequential Logic (`seq`) **13**
10.1 Basic Clock 13
10.2 Clock with Asynchronous Reset 13
10.3 Falling-Edge Clocks and Interface Events 13
10.4 Automatic Blocking-to-Nonblocking Conversion 14

Chapter 11 Combinational Logic (`comb`) **15**

Chapter 12 Initial Blocks **16**

Chapter 13 Case Statements **17**
13.1 Basic Case . . . 17
13.2 Casez with Wildcards . . . 17
13.3 Multi-Pattern Items . . . 18

Chapter 14 Inside Operator **20**

Chapter 15 Compound Assignments **21**
15.1 Example . . . 21

Chapter 16 Control Flow **23**
16.1 If / Else . . . 23
16.2 For Loop . . . 23
16.3 For-Loop Step Variants . . . 23
16.4 Restrictions . . . 24

Chapter 17 Include Directive **25**
17.1 Basic Usage . . . 25
17.2 Example: Sharing defun Libraries . . . 25
17.3 Recursive Includes . . . 25
17.4 Restrictions . . . 26

Chapter 18 Module Instantiation **27**
18.1 Basic Instantiation . . . 27
18.2 Same-Name Shorthand . . . 27
18.3 Parameter Override . . . 27
18.4 Dangling Ports (_) . . . 27
18.5 Regex Port Mapping . . . 27
18.6 Port Connection Constants . . . 30

Chapter 19 Metaprogramming **32**
19.1 `` `for `` Loop . . . 32
19.2 `` `if `` Conditional . . . 34
19.3 Restrictions . . . 35

Chapter 20 Expressions and Operators **36**
20.1 Operator Precedence . . . 36
20.2 Bit Selection and Part Select . . . 36
20.3 Concatenation and Replication . . . 36
20.4 Type Casting . . . 37
20.5 Ternary Operator . . . 37

Chapter 21 Special Constants **38**
21.1 Fill Constants . . . 38
21.2 Width-Prefixed Literals . . . 38
21.3 Bare Integers . . . 38

Chapter 22 Types, RTL Functions and Assertions **39**
22.1 Enums, Aliases and Packed Structs . . . 39
22.2 Synthesizable RTL Functions . . . 39
22.3 Static and Runtime Assertions . . . 40

Chapter 23 Signal Bundles **41**

Chapter 24 IP Integration **42**
24.1 How It Works . . . 42
24.2 Filelist Support . . . 42
24.3 Two-Pass Scanning . . . 42
24.4 Example . . . 43

Chapter 25 Native SystemVerilog Interfaces **44**
25.1 External Definition . . . 44
25.2 Instances and Modport Ports . . . 44
25.3 Field Access and Whole-Group Connections . . . 45
25.4 Complete Parameterized Example . . . 45
25.5 Inspection and Current Scope . . . 46

Chapter 26 Semantic Rules **47**
26.1 Multi-Driver Detection . . . 47
26.2 Combinational Loop Detection . . . 47
26.3 Width Mismatch Warnings . . . 48
26.4 RTL Analysis and Connection Warnings . . . 48
26.5 Localparam Override Protection . . . 49
26.6 Implicit Signal Width Inference . . . 49
26.7 Generated Code Annotations . . . 49
26.8 Integer Literal Validation . . . 50

Chapter 27 Error Reference **51**
27.1 Compilation Errors . . . 51
27.2 Warnings . . . 53

Chapter 28 Complete Examples **54**
28.1 Combined RTL Features . . . 54
28.2 Parameterized Pipeline Register . . . 55
28.3 Bit Reversal with Metaprogramming . . . 56
28.4 State Machine with Localparams . . . 57
28.5 Arbiter with Case Statement . . . 59

28.6 Hierarchical Design 61

Appendix A Formal Grammar (EBNF) 64

Chapter 1 Introduction

Slip is a minimal hardware description language (DSL) that compiles `.slip` source files into synthesizable SystemVerilog. It is designed as syntactic sugar over SystemVerilog, providing a streamlined syntax for common hardware design patterns while generating clean, human-readable, IEEE 1800-2017 compliant output.

1.1 Design Philosophy

- **Minimal syntax** — port directions, signal types, and instance wiring are inferred from context.
- **Readable output** — RTL maps to SystemVerilog constructs; metaprograms and bundles expand into ordinary RTL, while native interfaces retain their SV form.
- **Compile-time metaprogramming** — `` `for `` loops and `` `if `` conditionals are fully expanded before code generation, with SystemVerilog constant evaluation.
- **Validated output** — generated SystemVerilog is checked against the IEEE 1800-2017 standard via slang.

Chapter 2 Installation

2.1 Prerequisites

- A C++20 compiler and CMake $\geq$ 3.24
- PCRE2 8-bit development libraries
- slang 10, installed or fetched by CMake from the pinned source revision
- Python only when building slang's generated C++ headers from source

The Slip executable does not require Python or pyslang at runtime. The Python implementation is maintained on the `python-reference` branch.

2.2 Build and Install

```
git clone https://github.com/himingway/slip.git
cd slip
cmake --preset release
cmake --build --preset release --parallel 2
ctest --preset release
cmake --install build/release --prefix "$HOME/.local"
```

Add `$HOME/.local/bin` to `PATH` to use `slip`. Offline builds can use `CMAKE_PREFIX_PATH` and `SLIP_FETCH_SLANG=OFF`; see `docs/development.md`. Verilator enables the independent RTL behavioral test.

Chapter 3 Command Line Interface

3.1 `slip build` — Compile to SystemVerilog

```
slip build <file.slip>                  # output to ./build/
slip build -o <out_dir> <file.slip>     # custom output directory
slip build -ip <ip_dir> <file.slip>     # include external IP directory (recursive)
slip build -ip <dir1> -ip <dir2> <file.slip> # multiple IP directories
slip build -f <file.f> <file.slip>      # use VCS-format filelist
slip build -f <file.f> -ip <dir> <file.slip> # filelist + directory (combined)
```

Each module in the `.slip` file generates a separate `.sv` file in the output directory.

3.2 `slip check` — Validate without generating output

```
slip check <file.slip>                  # validate only
slip check -ip <ip_dir> <file.slip>     # with IP directory
slip check -f <file.f> <file.slip>      # with filelist
```

Runs the full compilation pipeline but discards the output. Generated modules and external IP are validated together, sharing the same IP preprocessing context. All modules are validated before any build output is written.

Use `--strict` with either command in CI to require explicit signal and port declarations and reject assignment or port width mismatches. Default mode continues to support annotated implicit signals.

3.3 Project Configuration

With no input argument, the CLI reads `slip.conf` from the current directory. It does not search parent directories. For example:

```
entry = rtl/top.slip
out_dir = build/rtl
filelist = ip.f
ip_dir = "external ip"
strict = true
```

```
slip build
slip check --project examples/native-interface/slip.conf
slip inspect --project examples/native-interface/slip.conf --format=json
```

The format is one `key = value` per line, with `#` comments and unquoted, single-quoted or double-quoted paths. A path containing `#` must be quoted. Double quotes allow escaped quotes and backslashes; single quotes preserve backslashes literally. Configuration paths are relative to the file's directory. The default output is its `build` subdirectory.

Supported keys are `entry`, `out_dir`, `filelist`, `ip_dir`, `strict`, `no_implicit` and `widths_as_errors`. Boolean values are `true` or `false`. Only `filelist` and `ip_dir` may repeat; unknown keys, repeated scalar keys and invalid values report `E_CONFIG`. Without `entry`, supply the input on the command line.

Explicit input files do not load configuration unless `--project` is given. CLI input and `-o` override configuration; `-ip` and `-f` append to the configured lists. CLI paths remain relative to the working directory. Independent configuration checks override the `strict` shorthand regardless of key order. CLI checking flags apply in order: `--strict`, `--no-strict`, `--no-implicit`, `--allow-implicit`, `--widths-as-errors` and `--widths-as-warnings`.

3.4 Inspection and Machine-Readable Diagnostics

```
slip inspect design.slip
slip inspect --format=json design.slip
slip check --diagnostics=json design.slip
slip check -                    # read source from standard input
slip build --stdout design.slip # print SV instead of writing files
```

Inspection runs the complete compilation pipeline. It reports resolved ports, signals, parameters, instances, widths and connection origins. JSON adds `bundles`, `interfaces` and signal `type` information. A custom type's empty packed range does not mean it is one bit. Native interface field types reference `type(instance.field)`, and their width expressions use `$bits(instance.field)` for the total field size, including array elements. Diagnostics go to standard error; JSON inspection goes to standard output. Diagnostics include source locations and metaprogram expansion bindings where available.

Exit codes are 0 for success, 1 for compilation or I/O failure, and 2 for invalid CLI arguments or malformed project configuration.

Chapter 4 Language Overview

A Slip source file can contain modules, includes, generation-time `defun` definitions, shared types, RTL functions and bundle schemas. Comments use `//` (line) or `/* */` (block) syntax.

```
// This is a comment
module AND_gate (a, b, y) {
    logic a;
    logic b;
    logic y;
    assign y = a & b;
}
```

Generates:

```
module AND_gate (
    input logic a,
    input logic b,
    output logic y
);

    assign y = a & b;

endmodule
```

4.1 Key Differences from SystemVerilog

- Port directions are inferred by default; explicit `input`, `output` and `inout` declarations are also supported.
- Storage defaults to `logic`; `bit`, `int`, enums, aliases and packed structs are available.
- Blocking assignment in `seq` blocks is automatically converted to non-blocking.
- Compile-time metaprogramming via `` `for `` and `` `if ``.
- Template identifiers embed loop variables in signal names (e.g., `data_`i`).
- `include "file.slip"` shares modules, generation functions, types, RTL functions and bundle schemas across files.
- Bundles expand to ordinary ports; external SV interfaces support parameterized instances, modport ports and whole-group connections.
- Generated SV annotated with `// implicit`, `// no width`, `// width from X.Y` for easy review.

Chapter 5 Module Declaration

5.1 Basic Syntax

```
module <name> (port1, port2, ...) {
    // body
}
```

5.2 With Parameters

```
module <name> #(param W = 8, param DEPTH = 16) (port1, port2, ...) {
    // body
}
```

5.3 Implicit Ports

If the port list is omitted entirely, all identifiers used in the module body that are not declared internally become ports:

```
module passthrough {
    assign out = in;
}
```

Generates:

```
module passthrough (
    input logic in,
    output logic out
);

    assign out = in;

endmodule
```

Port directions are inferred from usage (see Chapter 8).

Chapter 6 Parameters and Localparams

6.1 Parameters (`param`)

Parameters are declared in the module header and can be overridden during instantiation unless they control Slip compile-time expansion:

```
module counter #(param WIDTH = 8, param MAX = 255) (clk, rst_n, count) {
    logic clk;
    logic rst_n;
    logic [WIDTH-1:0] count;

    seq (clk, neg: rst_n) {
        if (!rst_n) {
            count <= 0;
        } else if (count == MAX) {
            count <= 0;
        } else {
            count <= count + 1;
        }
    }
}
```

Generates:

```
module counter #(
    parameter WIDTH = 8,
    parameter MAX = 255
) (
    input logic clk,
    input logic rst_n,
    output logic [WIDTH - 1:0] count
);

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            count <= 0;
        end else begin
            if (count == MAX) begin
                count <= 0;
            end else begin
                count <= count + 1;
            end
        end
    end

endmodule
```

Override at instantiation:

```
module top (clk, rst_n, count) {
    logic clk;
    logic rst_n;
    logic [15:0] count;
    counter #(.WIDTH(16), .MAX(65535)) u_cnt { .clk, .rst_n, .count };
}
```

Generates:

```
module top (
    input logic clk,
    input logic rst_n,
    output logic [15:0] count
);

    counter #(
        .WIDTH(16),
        .MAX(65535)
    ) u_cnt (
        .clk(clk),
        .rst_n(rst_n),
        .count(count)
    );

endmodule
```

6.2 Localparams (localparam)

Localparams are declared in the module body and **cannot** be overridden during instantiation:

```
module fsm (clk, state) {
    logic clk;
    logic [1:0] state;
    localparam IDLE = 0;
    localparam RUN = 1;
    localparam DONE = 2;
}
```

Generates:

```
module fsm (
    input logic clk,
    input logic [1:0] state
);

    localparam IDLE = 0;
    localparam RUN = 1;
    localparam DONE = 2;

endmodule
```

Attempting to override a localparam in an instance raises a semantic error:

```
fsm #(.IDLE(3)) u1 { .clk, .state }; // ERROR
```

Localparams can be used in signal widths, `` `for `` bounds, and `` `if `` conditions.

Chapter 7 Signals

7.1 Explicit Declaration

```
logic [7:0] data;       // 8-bit signal
logic flag;             // 1-bit signal
signed [15:0] val;      // signed 16-bit
logic [7:0] mem [0:15]; // array of 16 x 8-bit
logic [7:0] matrix [0:3][0:15]; // multiple unpacked dimensions
logic [WIDTH-1:0] payload; // parameterized width
```

The `logic` keyword is optional for bare declarations:

```
data;                   // equivalent to: logic data;
```

Port names remain listed in the module header. Directions may be specified on their declarations, for example `input logic [7:0] data;` or `output packet_t packet;`. Driving an explicit input, including through an instance output or inout port, reports `E_DIRECTION`. Explicit scalar declarations retain their one-bit width rather than widening on connection.

7.2 Implicit Declaration

Signals that are referenced but not declared are automatically created as `logic` signals:

```
module example (a, b) {
    logic a;
    logic b;
    assign y = a & b; // 'y' is implicitly declared as logic
}
```

Generates:

```
module example (
    input logic a,
    input logic b
);

    logic y;

    assign y = a & b;

endmodule
```

Implicit signals used in instance port connections automatically inherit the target port's width, annotated with `// width from X.Y` in the generated SV. Other implicit signals default to 1-bit and are annotated with `// implicit, no width` for easy review.

Chapter 8 Port Inference

Slip infers port directions automatically from signal usage within the module body:

Usage Pattern	Inferred Direction
Only appears in expressions (right-hand side)	`input`
Only appears as assignment target (left-hand side)	`output`
Both read and driven	`output`

An instance inout connection can infer `inout`. Explicit direction declarations take precedence over inference.

8.1 Example

```
module adder (a, b, sum) {
    logic a;
    logic b;
    logic sum;
    assign sum = a + b;
}
```

Generates:

```
module adder (
    input logic a,
    input logic b,
    output logic sum
);

    assign sum = a + b;

endmodule
```

Chapter 9 Assignments

9.1 Continuous Assignment

At module level, both forms generate `assign` statements in SystemVerilog:

```
assign y = a & b;       // explicit 'assign' keyword
y = a + b;              // bare assignment (also generates 'assign')
```

Both generate continuous assignments:

```
assign y = a & b;
assign z = a + b;
```

9.2 Procedural Assignment

Inside `seq` and `comb` blocks, assignments become procedural:

```
seq (clk) {
    q <= d;             // non-blocking (sequential)
}

comb {
    y = sel ? a : b;    // blocking (combinational)
}
```

Generates:

```
    always_ff @(posedge clk) begin
        q <= d;
    end

    always_comb begin
        y = sel ? a : b;
    end
```

Remark The `assign` keyword must **not** be used inside `seq` or `comb` blocks. It is reserved for module-level continuous assignments only.

Chapter 10 Sequential Logic (`seq`)

The `seq` block generates an `always_ff` block with the specified clock and optional asynchronous reset.

10.1 Basic Clock

```
seq (clk) {
    q <= d;
}
```

Generates:

```
    always_ff @(posedge clk) begin
        q <= d;
    end
```

10.2 Clock with Asynchronous Reset

```
seq (clk, neg: rst_n) {
    if (!rst_n) {
        q <= 0;
    } else {
        q <= d;
    }
}
```

Generates:

```
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            q <= 0;
        end else begin
            q <= d;
        end
    end
```

Reset polarity options:

- `neg: rst_n` — active-low reset (`negedge rst_n`)
- `pos: rst` — active-high reset (`posedge rst`)

10.3 Falling-Edge Clocks and Interface Events

```
seq (neg: clk, neg: rst_n) {
```

```
    if (!rst_n) { q = '0; }
    else { q = d; }
}
// Interface fields may also supply the clock and reset:
seq (tx.clk, neg: tx.rst_n) { tx.data = data; }
```

`seq(clk)` and `seq(pos:clk)` use a rising edge; `seq(neg:clk)` uses a falling edge. Reset edges are selected independently. For a synchronous reset, use a conditional inside a clock-only process.

10.4 Automatic Blocking-to-Nonblocking Conversion

All blocking assignments (=) inside `seq` blocks are automatically converted to non-blocking (<=). This prevents common sequential logic mistakes:

```
seq (clk) {
    q = d;  // automatically becomes q <= d;
}
```

Generates (blocking = converted to non-blocking <=):

```
    always_ff @(posedge clk) begin
        q <= d;
    end
```

The conversion is unconditional. A warning is emitted for the assignments whose meaning the conversion changes: those that read their own previous value (a self-referencing statement such as `for (...) { sum = sum + b; }`, or a variable written and then read again on the same path). Under non-blocking semantics the read observes the pre-clock value, so such code should be restructured (or moved to a `comb` block) if the updated value was intended. Standard register idioms — shift registers over arrays, state machines, gated counters — convert silently.

Chapter 11 Combinational Logic (`comb`)

The `comb` block generates an `always_comb` block. Assignments inside remain blocking:

```
comb {
    if (sel) {
        y = a;
    } else {
        y = b;
    }
}
```

Generates:

```
    always_comb begin
        if (sel) begin
            y = a;
        end else begin
            y = b;
        end
    end
```

If the combinational block infers a latch (e.g., incomplete assignment in an `if` without `else`), Slip emits `always_latch` without a sensitivity list:

```
always_latch begin
    if (en) begin
        q <= d;
    end
end
```

Chapter 12 Initial Blocks

The `initial` block generates an `initial` block. It is primarily used in testbenches for initialization sequences. Assignments inside remain blocking:

```
initial {
    clk = 0;
    rst_n = 0;
}
```

Generates:

```
    initial begin
        clk = 0;
        rst_n = 0;
    end
```

Chapter 13 Case Statements

Slip supports `case`, `casez`, and `casex` statements with a brace-based syntax.

13.1 Basic Case

```
case (sel) {
    2'b00: y = a;
    2'b01: y = b;
    2'b10, 2'b11: y = c;
    default: y = 0;
}
```

Generates:

```
    always_comb begin
        case (sel)
            2'b00: begin
                y = a;
            end
            2'b01: begin
                y = b;
            end
            2'b10, 2'b11: begin
                y = c;
            end
            default: begin
                y = 0;
            end
        endcase
    end
```

13.2 Casez with Wildcards

`casez` treats `?` as don't-care bits:

```
casez (opcode) {
    4'b1???: y = a;
    4'b01??: y = b;
    4'b001?: y = c;
    4'b0001: y = d;
    default: y = 0;
}
```

Generates:

```
    always_comb begin
```

```
        casez (opcode)
            4'b1???: begin
                y = a;
            end
            4'b01??: begin
                y = b;
            end
            4'b001?: begin
                y = c;
            end
            4'b0001: begin
                y = d;
            end
            default: begin
                y = 0;
            end
        endcase
    end
```

13.3 Multi-Pattern Items

A single case item can match multiple patterns, separated by commas:

```
localparam IDLE = 0;
localparam WAIT = 1;
localparam RUN = 2;

comb {
    case (state) {
        IDLE, WAIT: y = 0;
        RUN: y = 1;
        default: y = 0;
    }
}
```

Generates (assuming `IDLE=0`, `WAIT=1`, `RUN=2`):

```
localparam IDLE = 0;
localparam WAIT = 1;
localparam RUN = 2;

    always_comb begin
        case (state)
            IDLE, WAIT: begin
                y = 0;
            end
            RUN: begin
                y = 1;
            end
            default: begin
                y = 0;
```

```
            end
        endcase
    end
```

Chapter 14 Inside Operator

The `inside` operator tests set membership, following SystemVerilog 2009:

```
if (state inside {IDLE, RUN, DONE}) begin
    // state is one of IDLE, RUN, DONE
end
```

The right-hand side is a set literal `{v1, v2, ...}`. At compile time, if all values are constants, the expression evaluates to 0 or 1.

```
localparam IDLE = 0;
localparam RUN = 1;
localparam DONE = 2;

comb {
    if (state inside {IDLE, RUN, DONE}) {
        active = 1;
    } else {
        active = 0;
    }
}
```

Generates:

```
    localparam IDLE = 0;
    localparam RUN = 1;
    localparam DONE = 2;

    logic active;

    always_comb begin
        if (state inside {IDLE, RUN, DONE}) begin
            active = 1;
        end else begin
            active = 0;
        end
    end
```

Chapter 15 Compound Assignments

Slip supports all SystemVerilog compound assignment operators. They are desugared at parse time:

Operator	Desugars To	Example
`+=`	`a = a + b`	`i += 1`
`-=`	`a = a - b`	`count -= 1`
`*=`	`a = a * b`	`val *= 2`
`/=`	`a = a / b`	`val /= 2`
`%=`	`a = a % b`	`val %= N`
`&=`	`a = a & b`	`flags &= MASK`
`|=`	`a = a | b`	`flags |= BIT`
`^=`	`a = a ^ b`	`flags ^= BIT`
`<<=`	`a = a << b`	`val <<= 2`
`>>=`	`a = a >> b`	`val >>= 2`
`<<<=`	`a = a <<< b`	`val <<<= 1`
`>>>=`	`a = a >>> b`	`val >>>= 1`

Compound assignments are valid in all contexts where regular assignments are used (module-level, `comb`, `seq`, `initial`).

15.1 Example

```
module m (clk, val, delta) {
    logic clk;
    logic [7:0] val;
    logic [7:0] delta;

    seq (clk) {
        val += delta;
        val -= 1;
    }
}
```

Generates (compound assignments desugared, blocking converted to non-blocking):

```
module m (
    input logic clk,
    output logic [7:0] val,
    input logic [7:0] delta
);

    always_ff @(posedge clk) begin
        val <= val + delta;
        val <= val - 1;
    end
```

```
endmodule
```

Chapter 16 Control Flow

16.1 If / Else

```
if (condition) begin
    // ...
end else begin
    // ...
end
```

16.2 For Loop

Procedural `for` loops generate SystemVerilog `for` statements. They are valid inside `seq` and `comb` blocks:

```
seq (clk, neg: rst_n) {
    if (!rst_n) {
        for (i = 0; i < 4; i = i + 1) {
            reg_file[i] <= 0;
        }
    }
}
```

Generates:

```
    logic [7:0] reg_file [0:3];

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            for (int i = 0; i < 4; i = i + 1) begin
                reg_file[i] <= 0;
            end
        end
    end
```

Loop variables (e.g., `i`) are excluded from implicit signal creation — they are treated as loop-local integers.

16.3 For-Loop Step Variants

The step clause supports three forms:

```
for (i = 0; i < N; i = i + 1) { ... } // standard
for (i = 0; i < N; i += 1) { ... } // compound assignment
for (i = 0; i < N; i++)   { ... } // increment
```

Compound assignments (`+=`, `-=`, `*=`, etc.) are desugared at parse time: `i += 1` becomes `i = i + 1`. All three

forms generate the same SystemVerilog output:

```
for (int i = 0; i < N; i = i + 1) begin
    // ...
end
```

16.4 Restrictions

- `while` loops are not supported (combinational while loops are unsafe in synthesis).

Chapter 17 Include Directive

The `include` directive imports modules, `defun` definitions, shared types, RTL functions and bundle schemas from another `.slip` file. It must appear at the top level, outside module bodies.

17.1 Basic Usage

```
include "helpers.slip";
include "utils/common.slip";
```

Included files are lexed, parsed, and their contents merged into the compilation unit. Relative paths are resolved from the directory of the including file.

17.2 Example: Sharing defun Libraries

File `helpers.slip`:

```
defun prefix(s):
    return "bus_" + s

defun inc(n):
    return n + 1
```

File `design.slip`:

```
include "helpers.slip";

module top (clk) {
    logic clk;
    inner u1 {
        .clk,
        "data_(.*)" => "$prefix(\1)",
        "ch_(.*)" => "$inc(\1)"
    };
}
```

17.3 Recursive Includes

Include directives in included files are also resolved. The same file is never included twice (deduplicated by absolute path).

17.4 Restrictions

- `include` must appear at the top level — not inside module bodies.
- The included path must be a string literal. Variable-based paths are not supported.
- Circular includes are detected and silently skipped.

Chapter 18 Module Instantiation

18.1 Basic Instantiation

```
module_name instance_name {
    .port1(signal1),
    .port2(signal2)
};
```

18.2 Same-Name Shorthand

When the port name matches the signal name, the signal expression can be omitted:

```
inner u1 { .clk, .rst_n, .data_in };
// Equivalent to: .clk(clk), .rst_n(rst_n), .data_in(data_in)
```

18.3 Parameter Override

```
inner #(.WIDTH(16), .DEPTH(32)) u1 { .clk, .data_in, .data_out };
```

18.4 Dangling Ports (_)

Use _ to leave a port intentionally unconnected:

```
inner u1 { .clk, .rst_n(_), .data_in };
```

In the generated SystemVerilog, the `.rst_n()` connection is omitted entirely.

18.5 Regex Port Mapping

Use regex patterns to map groups of ports to signals:

```
child u1 {
    .clk,
    "data_(.*)" => "bus_\1"
};
```

This maps `data_0` → `bus_0`, `data_1` → `bus_1`, etc. Signal widths are inferred from the target module's port declarations.

Regex connections are processed in order and can be mixed with explicit connections. The priority order is:

1. Explicit connections (highest priority)
2. Same-name shorthand
3. Regex patterns (first matching rule wins)

18.5.1 Regex Functions

Use `$func(...)` syntax to transform captured groups in replacements:

```
child u1 {
    .clk,
    "data_(.*)" => "bus_$upper(\1)", // uppercase
    "ch_(.*)" => "port_$add(\1, 1)" // arithmetic
};
```

Built-in functions:

- **String**: `$upper`, `$lower`, `$reverse`, `$substr`, `$replace`, `$concat`
- **Arithmetic**: `$add`, `$sub`, `$mul`, `$div`, `$mod`
- **Bit**: `$bit_reverse`, `$bit_select`

The arithmetic functions require a numeric capture (a port named `ch_b` cannot be incremented; use a string function for non-numeric names). The failure is reported as a located error naming the port and instance.

Multiple functions and prefix/suffix text are supported:

```
"port_(.*)" => "pre_$upper(\1)_post" // prefix + function + suffix
"a_(.*)_b_(.*)" => "$upper(\1)_$lower(\2)" // multiple functions
```

18.5.2 Native Custom Functions (defun)

Define custom functions directly in Slip using Python-style syntax:

```
defun prefix(s):
    return "bus_" + s

defun to_upper(s):
    return s.upper()

defun inc(n):
    return n + 1

module top (clk) {
    logic clk;
    inner u1 {
        .clk,
        "data_(.*)" => "$prefix(\1)",
        "name_(.*)" => "$to_upper(\1)",
        "ch_(.*)" => "$inc(\1)"
    };
}
```

The C++ interpreter supports a restricted Python-style expression language, including:

- String concatenation: `a + b`
- String methods: `.upper()`, `.lower()`, `.reverse()`, `.replace()`, `.strip()`, `.split()`, `.startswith()`, etc.
- Arithmetic: `+`, `-`, `*`, `/`, `%`
- Method chaining: `s.upper().reverse()`
- Built-in functions: `len()`, `int()`, `str()`, `range()`, etc.
- Indexing and slicing: `s[0]`, `s[1:3]`, `s[3:]`, `s[:3]`, `s[-1]`
- Ternary expressions: `x if cond else y`

18.5.2.1 Examples

Replace characters in port names:

```
defun fix_names(s):
    return s.replace("_", "-")

module inner (data_in, data_out) {
    logic data_in;
    logic data_out;
    assign data_out = data_in;
}

module top (clk) {
    logic clk;
    inner u1 { .clk, "data_(.*)" => "bus_$fix_names(\1)" };
    // data_in -> bus_data-in, data_out -> bus_data-out
}
```

Extract part of port name using slicing:

```
defun shorten(s):
    return s[:4] // take first 4 characters

module inner (channel_a, channel_b) {
    logic channel_a;
    logic channel_b;
    assign channel_b = channel_a;
}

module top (clk) {
    logic clk;
    inner u1 { .clk, "channel_(.*)" => "$shorten(\1)" };
    // channel_a -> chan, channel_b -> chan
}
```

Combine methods with chaining:

```
defun clean_upper(s):
    return s.strip().upper().replace("_", "")
```

```
module inner (sig_in, sig_out) {
    logic sig_in;
    logic sig_out;
    assign sig_out = sig_in;
}

module top (clk) {
    logic clk;
    inner u1 { .clk, "sig_(.*)" => "port_$clean_upper(\1)" };
    // sig_in -> port_SIGIN, sig_out -> port_SIGOUT
}
```

Conditional logic with `startswith()`:

```
defun prefix_by_type(s):
    return "axi_" + s if s.startswith("ar") else "apb_" + s

module inner (araddr, awaddr, wdata) {
    logic araddr;
    logic awaddr;
    logic wdata;
    assign wdata = araddr;
}

module top (clk) {
    logic clk;
    inner u1 { .clk, "(.*)" => "$prefix_by_type(\1)" };
    // araddr -> axi_araddr, awaddr -> apb_awaddr, wdata -> apb_wdata
}
```

Functions can be defined anywhere at the top level (before or after modules). All `defun` functions are registered before instance resolution.

18.6 Port Connection Constants

Port connection expressions that are constants must follow these rules:

- **Allowed**: width-prefixed literals (e.g., `1'b0`, `8'hFF`) and fill constants (`'0`, `'1`).
- **Forbidden**: bare integers (e.g., `0`, `42`).

```
// WRONG -- bare integer in port connection
inner u1 { .data_in(0) };

// OK -- width-prefixed
inner u1 { .data_in(1'b0) };

// OK -- fill constant (width determined by target port)
inner u1 { .data_in('0) };
```

The fill constants `'0` and `'1` are SystemVerilog native adaptive-width constants. Slip passes them through

directly; the downstream tool determines their width from the target port.

Chapter 19 Metaprogramming

Slip provides compile-time `` `for `` loops and `` `if `` conditionals. These are fully expanded before code generation.

19.1 `` `for `` Loop

Syntax uses backtick-prefixed keywords and loop variables:

```
`for (`i = 0; `i< 4; `i= `i+ 1) {
   logic [7:0] data_`i;
   assign data_`i = `i;
}
```

After expansion, this becomes:

```
logic [7:0] data_0;
logic [7:0] data_1;
logic [7:0] data_2;
logic [7:0] data_3;
assign data_0 = 0;
assign data_1 = 1;
assign data_2 = 2;
assign data_3 = 3;
```

19.1.1 Template Identifiers

Loop variables can appear inside identifier names using backtick syntax. The lexer merges the surrounding identifier text with the backtick variable into a single token:

```
`for (`i = 0; `i< 3; `i= `i+ 1) {
   assign out_`i = in_`i;
}
```

Generates:

```
   logic out_0;
   logic in_0;
   logic out_1;
   logic in_1;
   logic out_2;
   logic in_2;

   assign out_0 = in_0;
   assign out_1 = in_1;
   assign out_2 = in_2;
```

19.1.2 Nested Loops

```
`for(`i = 0; `i< 2; `i= `i+ 1) {
   `for(`j = 0; `j< 2; `j= `j+ 1) {
      assign result_`i_`j = `i+ `j;
   }
}
```

Generates:

```
   logic result_0_0;
   logic result_0_1;
   logic result_1_0;
   logic result_1_1;

   assign result_0_0 = 0;
   assign result_0_1 = 1;
   assign result_1_0 = 1;
   assign result_1_1 = 2;
```

Template identifiers with multiple backtick variables (e.g., `a`i_`j`) are fully supported. The lexer merges them into a single token in a single pass.

19.1.3 Constant Folding

Loop variables are substituted into hardware expressions without changing literal widths, signedness, four-state values, or assignment context. Hardware arithmetic is left to SystemVerilog. Pure decimal constants in index and dimension expressions can be folded; algebraic identities such as replacing an unknown value multiplied by zero are not applied.

Compile-time controls are evaluated by slang with SystemVerilog semantics. When Slip instances override parameters that control generation, the compiler creates a specialized module for each effective generation binding. Identical bindings reuse the same module; names are deterministic. Generation overrides are consumed by Slip, while ordinary RTL parameter overrides remain in SV. Each emitted module retains elaboration checks that prevent later SV overrides from contradicting its expanded structure.

```
module top(a, b) {
   logic [1:0] a;
   logic [3:0] b;
   generated #(.N(2)) small_instance { .y(a) };
   generated #(.N(4)) large_instance { .y(b) };
}
```

Here `generated` denotes a module whose `N` parameter controls a generation loop or conditional. Binding dependencies propagate through nested wrappers and RTL function generation controls. Override values must be evaluable in the caller's constant environment. The default template is also compiled and validated. Limits are 1,024 specializations per compilation and a dependency depth of 128; generation integers retain the signed 64-bit limit.

Example — bit reversal with parameterized width:

```
module bit_reverse #(param W = 8) (din, dout) {
    logic [W-1:0] din;
    logic [W-1:0] dout;

    comb {
        `for(`i = 0; `i< W; `i= `i+ 1) {
            dout[W - 1 - `i] = din[`i];
        }
    }
}
```

When `W=8` and `` `i=0 ``, substitution produces `dout[W - 1 - 0] = din[0]`. Expressions containing `W` stay symbolic and are evaluated by the downstream tool.

19.1.4 Compile-Time Expression Evaluation

`` `for `` init, bound, and step expressions, as well as `` `if `` conditions, must be evaluable to integer constants at compile time. The evaluation engine supports:

- Integer literals and `param`/`localparam` references
- Arithmetic: `+`, `-`, `*`, `/`, `%`, `**`
- Relational: `<`, `>`, `<=`, `>=`, `==`, `!=`, `===`, `!==`
- Logical: `&&`, `||`
- Bitwise: `&`, `|`,`^`,`~`, `<<`, `>>`, `<<<`, `>>>`
- Unary prefix: `-`, `!`,`~`
- Parenthesized sub-expressions

19.2 `if Conditional

```
`if (ENABLE_DEBUG) {
    logic [7:0] debug_out;
    assign debug_out = internal_signal;
}
`else{
    assign debug_out = 0;
}
```

When `ENABLE_DEBUG` evaluates to non-zero, generates:

```
    logic [7:0] debug_out;
    assign debug_out = internal_signal;
```

When `ENABLE_DEBUG` evaluates to zero, generates:

```
assign debug_out = 0;
```

The condition must be a compile-time constant expression (parameter, localparam, or integer literal expression).

19.3 Restrictions

- Loop variables (`` `i ``) cannot be assigned within the loop body — they are controlled by the expansion engine.
- `` `for `` / `` `if `` conditions must be evaluable at compile time.
- The step variable must match the loop variable (e.g., `` `i = `i + 1 ``).
- A default maximum of 10,000 loop iterations across each module; repeated loop values are diagnosed immediately.

Chapter 20 Expressions and Operators

20.1 Operator Precedence

From highest to lowest (following IEEE 1800-2017):

Priority	Operators	Associativity
1 (highest)	Selections, fields, function calls and casts	Left
2	Unary arithmetic, logical and reduction operators	Right
3	`**`	Left
4	`*`, `/`, `%`	Left
5	`+`, `-` (binary)	Left
6	`<<`, `>>`, `<<<`, `>>>`	Left
7	`<`, `>`, `<=`, `>=`, `inside`	Left
8	`==`, `!=`, `===`, `!==`	Left
9	`&`	Left
10	`^`	Left
11	`|`	Left
12	`&&`	Left
13	`||`	Left
14 (lowest)	`?:` (ternary)	Right

20.2 Bit Selection and Part Select

```
data[3]        // bit select
data[7:0]      // part select
data[i]        // variable index
data[base +: 8] // 8 bits starting at base, ascending
data[15 -: 8]  // 8 bits starting at bit 15, descending
```

Indexed-select widths must be constant under SV rules; the starting index may vary at runtime. Bounds and width diagnostics use the elaborated SV types.

20.3 Concatenation and Replication

```
{a, b, c}      // concatenation
{4{1'b1}}      // replication: 4 copies of 1'b1
```

Nested replications are supported: `{4{{2{1'b0}}}}` produces 8 bits of zero.

Concatenations can also be assignment targets:

```
assign {hi, lo} = data;
```

```
assign word[15 -: 8] = byte_value;
```

Targets may combine identifiers, struct fields, array elements, part selects and parentheses. Driver checks distinguish fields and constant disjoint ranges; dynamic selections use conservative overlap checks.

20.4 Type Casting

```
signed'(expr)  // signed cast
unsigned'(expr) // unsigned cast
8'(expr)       // width cast (truncate or zero-extend to 8 bits)
packet_t'(expr) // cast to a declared type
```

Both `signed` and `unsigned` without a following `'` raise a syntax error.

20.5 Ternary Operator

```
sel ? a : b
```

Right-associative: `a ? b : c ? d : e` is parsed as `a ? b : (c ? d : e)`.

Chapter 21 Special Constants

21.1 Fill Constants

```
'0   // all bits zero (width determined by context)
'1   // all bits one (width determined by context)
```

These are SystemVerilog native fill constants. Their width is automatically determined by the connected port or assignment target. Slip passes them through without width inference.

21.2 Width-Prefixed Literals

```
1'b0        // 1-bit binary zero
1'b1        // 1-bit binary one
8'hFF       // 8-bit hexadecimal
4'b1010     // 4-bit binary
16'd255     // 16-bit decimal
8'o377      // 8-bit octal
```

Digits are validated against the specified radix. For example, `4'bABCD` and `4'o888` are rejected. The four-state values `x` and `z` are accepted in any radix.

21.3 Bare Integers

```
0           // bare integer (OK in assignments and expressions)
42          // bare integer
```

Bare integers are **not allowed** in port connections — use width-prefixed literals or fill constants instead.

Chapter 22 Types, RTL Functions and Assertions

22.1 Enums, Aliases and Packed Structs

```
typedef enum logic [1:0] { IDLE, RUN = 2, DONE } state_t;
typedef logic [7:0] byte_t;
typedef struct packed {
    byte_t data;
    logic valid;
} packet_t;

module decode(input_packet, output_packet) {
    input packet_t input_packet;
    output packet_t output_packet;
    state_t state;
    assign output_packet.data = input_packet.data;
    assign output_packet.valid = input_packet.valid;
}
```

Types may be declared at the top level or inside a module. Top-level declarations, including those loaded by `include`, are copied into each module's scope. User-defined port types use non-ANSI SV declarations so the typedefs are in scope. Packed structs support data fields and nested declared types. Field selection participates in assignment, driver and combinational-feedback checks.

Enum values may be explicit or advance from the preceding value. The default base type is SV `int`. An inline declaration such as `enum [1:0] { IDLE, RUN } state;` creates an anonymous type and variable. `typedef logic [7:0] byte_t;` defines a packed alias.

22.2 Synthesizable RTL Functions

```
function [7:0] invert(logic [7:0] x) {
    logic [7:0] tmp;
    tmp = ~x;
    if (x == 0) { return 8'hff; }
    else { return tmp; }
}
module example(a, y) {
    logic [7:0] a;
    logic [7:0] y;
    assign y = invert(a);
}
```

RTL functions may appear at the top level or inside modules and emit `function automatic`. Return types can be `logic`, `bit`, `int`, a packed width or a named type; an omitted type means scalar logic. Arguments are inputs. Local variables, types, constants, control flow, generation loops, assertions and returns are supported.

Functions may read their arguments, local names and module constants. Assignments must be blocking, and all control paths must return a value. Nonlocal signal dependencies, instances, nested functions and clocked processes are rejected with `E_FUNCTION`. Local names do not become module ports. Generation-time `defun` functions remain a separate mechanism for producing connection names and constant values.

22.3 Static and Runtime Assertions

```
module control #(param DEPTH = 8) (clk, valid) {
    logic clk;
    logic valid;
    static_assert(DEPTH > 0, "DEPTH must be positive");
    seq (clk) { assert(valid, "valid required"); }
}
```

`static_assert` belongs at module level. It is checked during compilation and retained as an SV elaboration check, so ordinary instance parameter overrides must also satisfy the constraint. Conditions can use constant expressions, module constants, type queries and evaluable RTL functions. Direct evaluation failures report `E_ASSERT`; failures found during SV elaboration report `E_SV`.

`assert` belongs inside a process or RTL function. It emits an immediate assertion followed by `$error` on failure. Messages are optional. Enable assertions in the downstream simulator, for example with Verilator's `--assert` flag.

Chapter 23 Signal Bundles

```
bundle Stream #(param W = 8) {
    output logic [W-1:0] data;
    output logic valid;
    input logic ready;
}

module passthrough(rx, tx) {
    slave Stream #(.W(16)) rx;
    master Stream #(.W(16)) tx;
    assign tx.data = rx.data;
    assign tx.valid = rx.valid;
    assign rx.ready = tx.ready;
}
```

Directions in the schema use the master's viewpoint; a slave reverses them. Exported groups must appear in the module's port list and declare a `master` or `slave` role. Internal groups use `bus Stream link;`. Schema parameters use named overrides, with validation of unknown names, duplicates and dependent defaults.

Fields expand to ordinary SV signals and ports, such as `rx_data` and `rx_ready`. A connection like `.rx(link)` expands the complete group; `.rx` connects a group named `rx`. Both ends must have the same schema and fields. Field widths retain full SV checks. Explicit field connections and regex connections remain available. Unknown fields, conflicting directions, duplicate drivers and parameter errors are diagnosed.

`inspect` lists schemas and fields; JSON exposes `bundles` and marks expanded connection origins as `bundle`.

Chapter 24 IP Integration

Slip can instantiate external SystemVerilog modules by providing IP directories or VCS-format filelists:

```
slip build -ip ./ip_cores -ip ./vendor_lib design.slip
slip build -f ip.f design.slip
slip build -f ip.f -ip ./ip_cores design.slip
```

24.1 How It Works

1. Slip collects IP sources from filelists (`-f`) and IP directories (`-ip`).
2. IP directories are scanned recursively for `.sv` and `.v` files.
3. All IP files are parsed together with a shared preprocessor context, so `` `define `` macros propagate across files.
4. When an instance references a module not defined in the `.slip` source, Slip uses slang to reflect the external module's ports and parameters.
5. Port widths are evaluated using the instance's parameter overrides.
6. Signals are created with correct widths for regex-mapped connections.

24.2 Filelist Support

Slip supports VCS-format filelist (`.f`) files for explicit control over IP source ordering, include paths, and macro definitions:

```
// ip.f
+incdir+./include
+define+DATA_WIDTH=32
+define+ADDR_WIDTH=8
./rtl/fifo.sv
./rtl/arbiter.sv
-f vendor_lib.f
```

- `+incdir+path` — include search directory (multiple paths separated by `+`)
- `+define+MACRO=value` — macro definition
- `-f sub.f` — nested filelist reference (recursive, with circular reference detection)
- File paths are resolved relative to the filelist location
- Duplicate files are automatically deduplicated
- Comments (`//`) and empty lines are ignored

24.3 Two-Pass Scanning

IP scanning uses a two-pass approach to resolve `` `define `` ordering:

Pass 1 — Collection: Parse all filelists, collect source files, include paths, and macro definitions. Recursively scan IP directories and merge with filelist entries (deduplication).

Pass 2 — Parsing and indexing: Parse all IP files together using slang's `SyntaxTree::fromFiles()` with a shared preprocessor context. This preserves macro ordering across files. Ports and parameters are reflected lazily when Slip actually references a module or interface.

An unused module with unsupported type parameters no longer blocks the whole IP directory merely because reflection cannot represent it. Referenced modules with type parameters still report `E_IP`. Supported native interface ports are described in Chapter 25. Syntax errors in external files and semantic errors in the actual design are still reported. Validation selects roots from the generated design instead of elaborating unrelated vendor roots.

24.4 Example

Given an external IP file `ip_cores/fifo.sv`:

```
module fifo #(
    parameter DEPTH = 16,
    parameter WIDTH = 8
)(
    input logic           clk,
    input logic           rst_n,
    input logic [WIDTH-1:0] din,
    output logic [WIDTH-1:0] dout,
    output logic          full
);
```

In your Slip source:

```
module top (clk, rst_n, data_in, data_out) {
    logic clk;
    logic rst_n;
    logic [7:0] data_in;
    logic [7:0] data_out;

    fifo #(.DEPTH(32)) u_fifo {
        .clk,
        .rst_n,
        .din(data_in),
        .dout(data_out),
        .full(_)
    };
}
```

The `.full(_)` connection leaves the `full` output dangling.

Chapter 25 Native SystemVerilog Interfaces

External SV interface definitions can be loaded using `-ip`, `-f` or project configuration. Slip emits real interface instances and modport ports. This facility is distinct from the ordinary signal expansion of bundles (Chapter 23); there is no implicit conversion between the two.

25.1 External Definition

The example interface below has a width parameter, clock/reset constructor ports and master/slave modports. The external sink uses an interface port and checks that its own width parameter matches the actual interface.

Listing 25.1: External Stream interface and sink

```
interface Stream #(parameter int W = 8) (input logic clk, rst_n);
    logic [W-1:0] data;
    logic valid, ready;
    modport master(input clk, rst_n, ready, output data, valid);
    modport slave(input clk, rst_n, data, valid, output ready);
endinterface

module stream_sink #(parameter int W = 8) (
    Stream.slave bus,
    output logic [W-1:0] observed
);
    assign bus.ready = 1'b1;
    assign observed = bus.valid ? bus.data : '0;
    if ($bits(bus.data) != W) begin : width_check
        $error("stream_sink W must match its interface data width");
    end
endmodule
```

25.2 Instances and Modport Ports

```
// Internal interface instance, including constructor connections:
interface Stream #(.W(16)) link { .clk(clk), .rst_n(rst_n) };

// Inside a module whose port list includes tx:
interface Stream.master tx;
```

An internal declaration accepts named parameter overrides and named constructor connections; `.clk` is shorthand for `.clk(clk)`. With no constructor connections, the braces may be omitted. Unknown or duplicate parameters and localparam overrides are rejected.

Slip interface ports must appear in the module header and specify a modport. They inherit interface parameters and constructor connections from the caller's actual instance; neither can be specified again on the port

declaration. Internal instances are declared without a modport; select a view at the endpoint.

25.3 Field Access and Whole-Group Connections

```
assign tx.valid = enable;
seq (tx.clk, neg: tx.rst_n) { tx.data = data; }

producer source { .tx(link), .data, .enable };
stream_sink sink { .bus(link.slave), .observed };
```

Fields support expressions, selects, arrays and packed-struct member access. Clock and reset expressions may also reference fields. Unknown or hidden fields report `E_INTERFACE`. Writing an input field, including via an instance output or inout connection, reports `E_DIRECTION`. Overlapping field drivers report `E_MULTIPLE_DRIVERS`. Slang analysis also checks driver conflicts involving the interface's own SV logic.

Whole-group connections accept a declared interface instance or port, optionally with an explicit modport selection. The interface type must match, and a selected modport must match the destination's restriction. A forwarded modport port keeps the same view, preserving direction and field-access restrictions. Missing interface connections, including explicit `_`, are errors. External unrestricted interface ports may receive internal instances; without a selected view they are treated as bidirectional connections.

The actual interface determines field types and widths. For example, connecting `Stream #(.W(16))` makes `tx.data` 16 bits. Use `static_assert($bits(tx.data) == W);` when a module parameter must match. Width mismatches are warnings by default and errors under `--strict`.

25.4 Complete Parameterized Example

Listing 25.2: Slip producer and native interface interconnect

```
module producer #(param W = 8) (tx, data, enable) {
    interface Stream.master tx;
    logic [W-1:0] data;
    logic enable;

    seq (tx.clk, neg: tx.rst_n) {
        if (!tx.rst_n) {
            tx.data = '0;
            tx.valid = 1'b0;
        } else if (tx.ready) {
            tx.data = data;
            tx.valid = enable;
        }
    }
}

module top #(param W = 16) (clk, rst_n, data, enable, observed) {
    logic clk;
```

```
    logic rst_n;
    logic [W-1:0] data;
    logic enable;
    logic [W-1:0] observed;

    interface Stream #(.W(W)) link { .clk, .rst_n };
    producer #(.W(W)) source {
        .tx(link), .data, .enable
    };
    stream_sink #(.W(W)) sink {
        .bus(link.slave), .observed
    };
}
```

The project configuration loads the external definition and enables strict checks:

Listing 25.3: Native interface project configuration

```
entry = design.slip
ip_dir = ip
out_dir = build/rtl
strict = true
```

From the repository root:

```
build/release/slip check --project examples/native-interface/slip.conf
build/release/slip build --project examples/native-interface/slip.conf
build/release/slip inspect --project examples/native-interface/slip.conf --format=json
```

Downstream tools must read both the generated modules and the external SV files. The compiler does not copy external definitions into each generated module.

25.5 Inspection and Current Scope

Text inspection lists the interface type, modport, parameters, instance/port identity and accessible fields. JSON adds a module-level `interfaces` array with `name`, `type`, `modport`, `port`, `location`, `parameters` and `fields`. Native fields also appear as dotted entries in `signals`; these checking-only entries do not generate extra declarations. Types use `type(instance.field)` and width expressions use `$bits(instance.field)` for the total field size. Whole-group connection origins are marked `interface`.

Supported definitions expose top-level data fields and modports composed of input/output/inout data fields. Current exclusions are interface instance/port arrays, nested interfaces, type parameters, clocking, modport expressions or renamed fields, ref fields and imported/exported tasks or functions. Unsupported views are rejected when used; unused complex modports do not block other supported views. The full syntax and boundaries are also documented in `docs/native-interfaces.md`.

Chapter 26 Semantic Rules

26.1 Multi-Driver Detection

A signal cannot be driven by multiple procedural blocks. This is caught at compile time:

```
// ERROR: signal 'q' driven by multiple blocks
seq (clk) { q <= a; }
seq (clk) { q <= b; }
```

The analyzer tracks assignments through `if` and `for` nesting within each block. A signal driven by two `seq`/`comb` blocks, or by a procedural block and a continuous assignment, produces an error.

The check is bit-range aware: two continuous assignments driving *disjoint* bits of the same variable are legal, which is what unrolled code such as:

```
`for (`i = 0; `i< 4; `i= `i+ 1) {
    assign mask[`i] = din[`i];
}
```

produces. Overlapping or whole-variable drivers are rejected.

26.2 Combinational Loop Detection

Feedback loops inside `comb` blocks are detected using DFS-based cycle detection on the signal dependency graph:

```
// ERROR: combinational loop detected
comb {
    a = b + 1;
    b = a + 1; // 'a' depends on 'b', 'b' depends on 'a'
}
```

The detector builds one dependency graph per module covering every `comb` block *and* every continuous assignment, so loops that cross a block boundary are caught as well.

Within a block, blocking (last-assignment-wins) semantics are modelled: a read resolves to the most recent prior assignment of what it reads, which is what keeps the standard *init-then-accumulate* idiom loop-free:

```
comb {
    y = 0;
    y = y + b; // OK: the read resolves to the 0 just assigned
}
```

Graph nodes are (signal, index) pairs, so storage written in disjoint elements stays independent. A chain over array elements is fine, while a genuine element cycle is reported with its indices:

```
assign st[0] = din; // OK
assign st[1] = st[0];
assign st[2] = st[1];

// ERROR: combinational loop detected: st[2] -> st[1] -> st[0] -> st[2]
```

If a cycle is found, compilation reports `E_COMB_LOOP`.

26.3 Width Mismatch Warnings

The compiler checks assignments and instance connections against their elaborated SV types. Width expansion, truncation and constant-conversion diagnostics become `W_WIDTH`, or `E_SV` under `--strict` or `--widths-as-errors`:

```
module m (out[7:0], other[3:0]) {
    assign out = other; // WARNING: 'out' is 8 bits, but value is 4 bits
}
```

Unsized decimal literals are range-checked against the target rather than assumed to be 32 bits, so the common `y = 0;` idiom stays silent while a value that does not fit warns:

```
assign out = 42; // OK: 42 fits in 8 bits
assign out = 300; // WARNING: value 300 does not fit in 'out' (8 bits)
```

Width-prefixed literals use their declared width:

```
module m (out[7:0]) {
    assign out = 8'hFF; // OK: both 8 bits
}
```

Instance port connections are also checked:

```
module sub (data[7:0]) { ... }
module top (out[3:0]) {
    sub u { .data(out) }; // WARNING: port expects 8 bits, signal is 4 bits
}
```

Widths are resolved in the actual SV parameterization, including native interface field types. Symbolic expressions remain in generated RTL for downstream tools; they are not discarded merely because the early reflection pass cannot reduce them.

26.4 RTL Analysis and Connection Warnings

After successful elaboration, slang dataflow analysis checks generated Slip RTL with external IP participating in driver and type resolution. Diagnostics include `W_UNDRIVEN` for internal signals without a driver and `W_LATCH` for incomplete combinational assignments. Complete cases, partial-bit assignments, loops, initialization and instance drivers are considered. Branch and loop analysis each have a 10,000-step resource bound and do not

replace simulation.

Regex rules that establish no connection report `W_REGEX_UNUSED`, including rules shadowed by explicit connections or earlier rules. Omitted ports report `W_UNCONNECTED`; explicit `.port(_)` marks an intentional omission. Omitted local input connections remain errors, and native interface connections are always required. These analysis and connection warnings remain warnings in strict mode.

26.5 Localparam Override Protection

Parameters can be overridden at instantiation; localparams cannot:

```
module example #(param W = 8) () {
    localparam MASK = 255;
}

// OK
example #(.W(16)) u1 {};

// ERROR: cannot override localparam 'MASK'
example #(.MASK(511)) u2 {};
```

26.6 Implicit Signal Width Inference

Implicit signals connected to instance ports automatically inherit the target port's width:

```
module top (a, b) {
    logic [7:0] a;
    logic [7:0] b;
    // 'sum' is implicit --- inherits [7:0] from adder.sum port
    adder u1 { .a, .b, .sum };
}
```

Generates:

```
    // width from adder.sum
    logic [7:0] sum;
```

Other implicit signals (not connected to ports) default to 1-bit and are annotated with `// implicit, no width` in the generated output.

26.7 Generated Code Annotations

Slip adds review annotations to generated SystemVerilog signals and ports:

These annotations help identify signals that may need explicit width declarations during code review. Use `grep "no width" *.sv` to quickly find all un-sized signals.

Annotation	Meaning
`// implicit`	Signal or port was auto-generated (not explicitly declared)
`// no width`	Signal or port has no declared width (defaults to 1-bit)
`// width from X.Y`	Width was automatically inferred from instance port

26.8 Integer Literal Validation

Verilog-style sized literals are validated at lex time:

- Digits must be valid for the specified radix (e.g., `4'bABCD` is rejected).
- The digit portion must not be empty or underscore-only (e.g., `4'b_` is rejected).
- Four-state values `x`/`z` are accepted in any radix.

Chapter 27 Error Reference

27.1 Compilation Errors

`cannot override localparam 'name' of module 'M' (instance 'I')`
Attempting to override a localparam in instance `#(...)`.

`unconnected input port 'name' in instance 'I' of module 'M'`
Input port not connected and not marked with `_`. The error includes active regex rules when applicable.

`unable to infer width for implicit signal 'name'`
Implicit signal used in a context where width cannot be determined (legacy; now defaults to 1-bit with `// implicit, no width` annotation).

`cyclic module dependency detected among: A, B`
Modules have circular instantiation dependency (e.g., module A instantiates B, and B instantiates A). This is illegal in synthesizable hardware.

`bare integer '...' in port connection must have explicit width`
Bare integer used in port connection. Use a width-prefixed literal (`1'b0`) or a fill constant (`'0`).

`signal 'name' driven by multiple drivers`
A signal is driven by more than one source (two procedural blocks, or a block and a continuous assignment) over overlapping bits. Disjoint bit-ranges are allowed.

`invalid digit 'X' for radix 'r' in literal`
Integer literal contains a digit invalid for the specified radix (e.g. `4'bABCD`).

`integer literal has no actual digit value`
Literal has only underscores after the radix prefix (e.g. `4'b_`).

`E_LIMIT`
A compilation resource limit was exceeded. Defaults include 10,000 metaprogram loop iterations and 100,000 expanded statements per module, 1,024 specializations per compilation and specialization dependency depth 128. Metaprogram controls and defun integers must fit a signed 64-bit integer; defun strings operate on bytes.

`E_CONFIG`
An unknown or repeated scalar project key, invalid boolean, or malformed quoted value. The diagnostic points to the configuration line.

`E_DIRECTION`
An explicit input signal or interface field is driven internally or through an instance output/inout connection, or modport views are incompatible.

`E_INTERFACE`
An unresolved native interface, incompatible type, unknown or inaccessible field, unsupported modport feature, or invalid interface declaration/connection. Slip interface ports require a modport and inherit configuration from the caller.

`E_BUNDLE`
An unknown bundle schema/field, incompatible group connection or invalid role.

`E_FUNCTION`
: An RTL function has nonlocal signal dependencies, stateful operations, nonblocking assignments, missing returns or incomplete return paths.

`E_ASSERT` / `E_SV`
: A static assertion failed during direct evaluation or SV elaboration, respectively. `E_SV` also reports other invalid generated or instantiated SV semantics.

`E_DUPLICATE`
: Conflicting declarations or repeated IP definitions. Duplicate IP modules are errors; no definition is chosen arbitrarily.

`cannot evaluate 'name' at compile time`
: Non-constant expression in `` `for ``/`` `if `` condition, init, or step.

`` `for step variable '...' must match loop variable '...' ``
: Step variable differs from the declared loop variable. The step expression determines the next value of the loop variable, so the step variable must match the loop variable.

`unsupported operator '...' in compile-time expression`
: Operator not supported by the constant expression evaluator.

`combinational loop detected: ...`
: A feedback loop was found. The message lists the storage involved, with indices where relevant (`st[2] -> st[1] -> st[0] -> st[2]`). The graph covers every `comb` block and every continuous assignment in the module.

`duplicate declaration of 'name'`
: A signal, instance, or localparam with the same name is declared more than once in the same module.

`duplicate module definition: 'name' is defined more than once`
: Two modules share a name in the compilation unit (including files pulled in by `include`).

`duplicate connection to port 'name' of instance 'I'`
: The same target port is connected twice in one instance.

`cannot resolve target module 'name' of instance 'I'`
: The instance target is neither defined in the compilation unit nor found in the IP index. Pass `-ip`/`-f` when the module is external IP; otherwise this is a typo.

`nonblocking assignment '<=' is not allowed at module level`
: `<=` outside a `seq` block. Use a `seq (clk) { ... }` block for sequential logic, or `=` for a continuous assignment.

`'if'/'for'/'case' is not supported at module level`
: A procedural statement at module level. Wrap it in `comb {}` or `seq (clk) {}`, or use the compile-time forms `` `if ``/`` `for ``.

`empty concatenation '{}' is not allowed`
: A concatenation with no operands has no width and no SystemVerilog equivalent.

`method '.name()' is not allowed in defun bodies`
: `defun` bodies are evaluated in a sandbox with a safe method allowlist.

`regex port mapping error for port 'name' of instance 'I'`
: Replacement evaluation failed: unknown `$func`, invalid backreference, or bad arguments.

`reset polarity inconsistency: ...`
: A reset signal is used with conflicting polarity (both `pos:` and `neg:`) across different `seq` blocks.

27.2 Warnings

`W_SEQ_CONVERSION`

A blocking assignment converts to nonblocking in a `seq` block (unconditionally, as described in the conversion chapter). This warning fires for the assignments whose meaning that changes: a read-modify-write of storage already written on the same path, such as a loop accumulator (`for (...) { sum = sum + b; }`) or `y = 0; y = y + b;`. The read will observe the pre-clock value, so restructure or use a `comb` block if the updated value was intended. Register transfers (shift registers, `t = a + 1; o = t;`) convert silently.

`width mismatch: 'name' is N bits, but value is M bits`

An assignment has mismatched widths between the target signal and the value expression.

`value N does not fit in 'name' (M bits)`

An unsized decimal literal is outside the target's range (e.g. `out = 300` for an 8-bit `out`).

`port width mismatch: 'port' expects N bits, but 'signal' is M bits`

An instance port connection has mismatched widths between the port declaration and the connected signal.

`unconnected output port 'name' in instance 'I' of module 'M'`

An output port matched no connection and no regex rule; it is left dangling.

`W_UNDRIVEN`, `W_LATCH`

An internal signal is undriven, or a combinational process infers a latch.

`W_REGEX_UNUSED`, `W_UNCONNECTED`

A regex rule made no connections, or an instance port was omitted without an explicit intentional omission.

`output directory ... contains SystemVerilog files not produced by this run`

Stale `.sv` files from an earlier build (a renamed module, say) are still present. Downstream tools that compile a whole directory would pick them up.

Chapter 28 Complete Examples

28.1 Combined RTL Features

`examples/features/slip.conf` enables strict compilation of a design that combines packed types, enums, RTL functions, master/slave bundles, parameter specialization, multidimensional arrays, indexed selects, concatenated targets, falling-edge processes and assertions. Inspect it from the repository root:

```
build/release/slip check --project examples/features/slip.conf
build/release/slip inspect --project examples/features/slip.conf
```

Listing 28.1: Combined RTL features

```
typedef struct packed {
    logic [3:0] hi;
    logic [3:0] lo;
} packet_t;
typedef enum logic [1:0] { IDLE, RUN } state_t;

function [7:0] swap_nibbles(logic [7:0] x) {
    packet_t value;
    value.hi = x[7:4];
    value.lo = x[3:0];
    return {value.lo, value.hi};
}

bundle Stream #(param W = 8) {
    output logic [W-1:0] data;
    output logic valid;
    input logic ready;
}

module pipeline #(param N = 1) (clk, rst_n, din, dout) {
    logic clk; logic rst_n;
    logic [7:0] din; logic [7:0] dout;
    logic [7:0] stages [0:N-1];
    static_assert(N > 0, "pipeline must have a stage");
    seq (neg: clk, neg: rst_n) {
        if (!rst_n) {
            `for(`i = 0; `i< N; `i= `i+ 1) { stages[`i] = 0; }
        } else {
            stages[0] = din;
            `for(`i = 1; `i< N; `i= `i+ 1) { stages[`i] = stages[`i-1]; }
        }
    }
    assign dout = stages[N-1];
}

module producer(din, tx) {
```

```
    logic [7:0] din;
    master Stream tx;
    assign tx.data = din;
    assign tx.valid = 1'b1;
}
module consumer(rx, dout) {
    slave Stream rx;
    logic [7:0] dout;
    assign dout = rx.data;
    assign rx.ready = 1'b1;
}
module features(clk, rst_n, din, base, hi, lo, selected, short_out, long_out, bus_out, active, packet
    ) {
    logic clk; logic rst_n;
    logic [7:0] din;
    logic [2:0] base;
    logic [3:0] hi; logic [3:0] lo; logic [3:0] selected;
    logic [7:0] short_out; logic [7:0] long_out; logic [7:0] bus_out;
    logic active;
    packet_t packet;
    state_t state;
    logic [7:0] memory [0:1][0:1];
    bus Stream link;

    assign packet.hi = din[7:4];
    assign packet.lo = din[3:0];
    assign memory[0][1] = swap_nibbles(din);
    assign {hi, lo} = memory[0][1];
    assign selected = din[base +: 4];
    pipeline #(.N(2)) short_pipe { .clk, .rst_n, .din(memory[0][1]), .dout(short_out) };
    pipeline #(.N(3)) long_pipe { .clk, .rst_n, .din(memory[0][1]), .dout(long_out) };
    producer p { .din, .tx(link) };
    consumer c { .rx(link), .dout(bus_out) };
    seq (neg: clk, neg: rst_n) {
        if (!rst_n) { state = IDLE; }
        else { if (din[0]) { state = RUN; } else { state = IDLE; } }
    }
    seq (neg: clk) { assert(base <= 4, "slice exceeds data width"); }
    assign active = state == RUN;
}
```

The native interface example and its external SV sources appear in Chapter 25.

28.2 Parameterized Pipeline Register

```
module pipeline_reg #(param WIDTH = 8) (clk, rst_n, d, q) {
    logic clk;
    logic rst_n;
    logic [WIDTH-1:0] d;
    logic [WIDTH-1:0] q;
```

```
    seq (clk, neg: rst_n) {
        if (!rst_n) {
            q <= 0;
        } else {
            q <= d;
        }
    }
}
```

Generates:

```
module pipeline_reg #(
    parameter WIDTH = 8
) (
    input logic clk,
    input logic rst_n,
    input logic [WIDTH - 1:0] d,
    output logic [WIDTH - 1:0] q
);

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            q <= 0;
        end else begin
            q <= d;
        end
    end

endmodule
```

28.3 Bit Reversal with Metaprogramming

```
module bit_reverse #(param W = 8) (din, dout) {
    logic [W-1:0] din;
    logic [W-1:0] dout;

    comb {
        `for(`i = 0; `i< W; `i= `i+ 1) {
            dout[W - 1 - `i] = din[`i];
        }
    }
}
```

Generates:

```
module bit_reverse #(
    parameter W = 8
) (
    input logic [W - 1:0] din,
    output logic [W - 1:0] dout
```

```
);

    if (W !== (8)) begin : __slip_frozen_W
        $error("W controls Slip compile-time expansion and cannot be overridden");
    end

    always_comb begin
        dout[W - 1 - 0] = din[0];
        dout[W - 1 - 1] = din[1];
        dout[W - 1 - 2] = din[2];
        dout[W - 1 - 3] = din[3];
        dout[W - 1 - 4] = din[4];
        dout[W - 1 - 5] = din[5];
        dout[W - 1 - 6] = din[6];
        dout[W - 1 - 7] = din[7];
    end

endmodule
```

Note: parameter-dependent indices remain symbolic. The downstream tool evaluates them during elaboration; W cannot be overridden because it controls compile-time expansion.

28.4 State Machine with Localparams

```
module arbiter (clk, rst_n, req, grant) {
    logic clk;
    logic rst_n;
    logic [3:0] req;
    logic [3:0] grant;
    logic [1:0] cur;

    localparam S_IDLE = 0;
    localparam S_GNT0 = 1;
    localparam S_GNT1 = 2;

    seq (clk, neg: rst_n) {
        if (!rst_n) {
            cur <= S_IDLE;
        } else {
            case (cur) {
                S_IDLE: {
                    if (req[0]) cur <= S_GNT0;
                    if (req[1]) cur <= S_GNT1;
                }
                S_GNT0: {
                    if (!req[0]) cur <= S_IDLE;
                }
                S_GNT1: {
                    if (!req[1]) cur <= S_IDLE;
                }
```

```
            default: cur <= S_IDLE;
        }
    }
}

assign grant[0] = (cur == S_GNT0);
assign grant[1] = (cur == S_GNT1);
}
```

Generates:

```
module arbiter (
    input logic clk,
    input logic rst_n,
    input logic [3:0] req,
    output logic [3:0] grant
);

    localparam S_IDLE = 0;
    localparam S_GNT0 = 1;
    localparam S_GNT1 = 2;

    logic [1:0] cur;

    assign grant[0] = (cur == S_GNT0);
    assign grant[1] = (cur == S_GNT1);

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            cur <= S_IDLE;
        end else begin
            case (cur)
                S_IDLE: begin
                    if (req[0]) begin
                        cur <= S_GNT0;
                    end
                    if (req[1]) begin
                        cur <= S_GNT1;
                    end
                end
                S_GNT0: begin
                    if (!req[0]) begin
                        cur <= S_IDLE;
                    end
                end
                S_GNT1: begin
                    if (!req[1]) begin
                        cur <= S_IDLE;
                    end
                end
                default: begin
                    cur <= S_IDLE;
                end
```

```
            endcase
        end
    end

endmodule
```

28.5 Arbiter with Case Statement

```
module arbiter_case (clk, rst_n, req, grant) {
    logic clk;
    logic rst_n;
    logic [3:0] req;
    logic [3:0] grant;
    logic [2:0] cur;

    localparam S_IDLE = 0;
    localparam S_GNT0 = 1;
    localparam S_GNT1 = 2;
    localparam S_GNT2 = 3;
    localparam S_GNT3 = 4;

    seq (clk, neg: rst_n) {
        if (!rst_n) {
            cur <= S_IDLE;
        } else {
            case (cur) {
                S_IDLE: {
                    if (req[0]) cur <= S_GNT0;
                    if (req[1]) cur <= S_GNT1;
                    if (req[2]) cur <= S_GNT2;
                    if (req[3]) cur <= S_GNT3;
                }
                S_GNT0: {
                    if (!req[0]) cur <= S_IDLE;
                }
                S_GNT1: {
                    if (!req[1]) cur <= S_IDLE;
                }
                default: cur <= S_IDLE;
            }
        }
    }

    comb {
        grant = 0;
        case (cur) {
            S_GNT0: grant[0] = 1;
            S_GNT1: grant[1] = 1;
            S_GNT2: grant[2] = 1;
            S_GNT3: grant[3] = 1;
```

```
            default: grant = 0;
        }
    }
}
```

Generates:

```
module arbiter_case (
    input logic clk,
    input logic rst_n,
    input logic [3:0] req,
    output logic [3:0] grant
);

    localparam S_IDLE = 0;
    localparam S_GNT0 = 1;
    localparam S_GNT1 = 2;
    localparam S_GNT2 = 3;
    localparam S_GNT3 = 4;

    logic [2:0] cur;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            cur <= S_IDLE;
        end else begin
            case (cur)
                S_IDLE: begin
                    if (req[0]) begin
                        cur <= S_GNT0;
                    end
                    if (req[1]) begin
                        cur <= S_GNT1;
                    end
                    if (req[2]) begin
                        cur <= S_GNT2;
                    end
                    if (req[3]) begin
                        cur <= S_GNT3;
                    end
                end
                S_GNT0: begin
                    if (!req[0]) begin
                        cur <= S_IDLE;
                    end
                end
                S_GNT1: begin
                    if (!req[1]) begin
                        cur <= S_IDLE;
                    end
                end
                default: begin
                    cur <= S_IDLE;
```

```
            end
        endcase
    end
end

always_comb begin
    grant = 0;
    case (cur)
        S_GNT0: begin
            grant[0] = 1;
        end
        S_GNT1: begin
            grant[1] = 1;
        end
        S_GNT2: begin
            grant[2] = 1;
        end
        S_GNT3: begin
            grant[3] = 1;
        end
        default: begin
            grant = 0;
        end
    endcase
end

endmodule
```

28.6 Hierarchical Design

```
module stage #(param W = 8) (clk, rst_n, data_in, data_out) {
    logic clk;
    logic rst_n;
    logic [W-1:0] data_in;
    logic [W-1:0] data_out;
    logic [W-1:0] reg_data;

    seq (clk, neg: rst_n) {
        if (!rst_n) {
            reg_data <= 0;
        } else {
            reg_data <= data_in;
        }
    }

    assign data_out = reg_data;
}

module pipeline (clk, rst_n, din, dout) {
    logic clk;
```

```
    logic rst_n;
    logic [7:0] din;
    logic [7:0] dout;
    logic [7:0] stage_data [0:3];

    assign stage_data[0] = din;

    `for(`i = 1; `i< 3; `i= `i+ 1) {
        stage #(.W(8)) u_stage_`i {
            .clk,
            .rst_n,
            .data_in(stage_data[`i - 1]),
            .data_out(stage_data[`i])
        };
    }

    assign dout = stage_data[2];
}
```

Generates:

```
module stage #(
    parameter W = 8
) (
    input logic clk,
    input logic rst_n,
    input logic [W - 1:0] data_in,
    output logic [W - 1:0] data_out
);

    logic [W - 1:0] reg_data;

    assign data_out = reg_data;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            reg_data <= 0;
        end else begin
            reg_data <= data_in;
        end
    end

endmodule

module pipeline (
    input logic clk,
    input logic rst_n,
    input logic [7:0] din,
    output logic [7:0] dout
);

    logic [7:0] stage_data [0:3];
```

```
    assign stage_data[0] = din;
    assign dout = stage_data[2];

    stage #(
        .W(8)
    ) u_stage_1 (
        .clk(clk),
        .rst_n(rst_n),
        .data_in(stage_data[0]),
        .data_out(stage_data[1])
    );

    stage #(
        .W(8)
    ) u_stage_2 (
        .clk(clk),
        .rst_n(rst_n),
        .data_in(stage_data[1]),
        .data_out(stage_data[2])
    );

endmodule
```

Appendix A Formal Grammar (EBNF)

```
CompilationUnit ::= { TopLevelItem }

TopLevelItem ::= ModuleDecl | FuncDef | IncludeDirective
            | TypeDecl | FunctionDecl | BundleSchema

IncludeDirective ::= "include" STRING ";"

ModuleDecl ::= "module" IDENT
             [ "#(" ParameterList ")" ]
             [ "(" PortList ")" ]
             "{" { Statement } "}"

ParameterList ::= ParameterDecl { "," ParameterDecl }
ParameterDecl ::= "param" IDENT "=" Expr

PortList    ::= PortItem { "," PortItem }
PortItem    ::= IDENT [ "[" Expr [ ":" Expr ] "]" ]

Statement ::= SignalDecl
           | LocalparamDecl
           | AssignStmt
           | SeqBlock
           | CombBlock
           | InitialBlock
           | IfStmt
           | ForStmt
           | CaseStmt
           | GenForStmt
           | GenIfStmt
           | InstanceStmt
           | BlockStmt
           | TypeDecl
           | FunctionDecl
           | ReturnStmt
           | AssertionStmt
           | BundleUse
           | NativeInterfaceUse

Direction ::= "input" | "output" | "inout"
BaseType  ::= "logic" | "bit" | "int" | IDENT
PackedWidth ::= "[" Expr [ ":" Expr ] "]"
UnpackedRange ::= "[" Expr ":" Expr "]"
TypedName ::= ["signed"] [BaseType] ["signed"]
              [PackedWidth] IDENT { UnpackedRange }
SignalDecl ::= [Direction] TypedName ";"

TypeDecl ::= "typedef" TypedName ";"
          | ["typedef"] EnumBody IDENT ";"
          | ["typedef"] StructBody IDENT ";"
```

```
EnumBody ::= "enum" [BaseType] ["signed"] [PackedWidth]
           "{" EnumItem { "," EnumItem } [","] "}"
EnumItem ::= IDENT [ "=" Expr ]
StructBody ::= "struct" "packed" "{" { SignalDecl } "}"

FunctionDecl ::= "function" ["automatic"] TypedName
               "(" [ FunctionArg { "," FunctionArg } ] ")"
               BlockStmt
FunctionArg ::= ["input"] TypedName
ReturnStmt ::= "return" Expr ";"
AssertionStmt ::= ("assert" | "static_assert")
                "(" Expr [ "," STRING ] ")" ";"

BundleSchema ::= "bundle" IDENT [ "#(" ParameterList ")" ]
               "{" { SignalDecl } "}" [";"]
NamedOverrides ::= "#(" [NamedParam { "," NamedParam }] ")"
BundleUse ::= ("master" | "slave" | "bus") IDENT
            [NamedOverrides] IDENT ";"
NativeInterfaceUse ::= "interface" IDENT [ "." IDENT ]
                    [NamedOverrides] IDENT
                    ["{" [NamedConnection
                       { "," NamedConnection }] "}"] ";"

FuncDef ::= "defun" IDENT "(" [IDENT { "," IDENT }] ")" ":"
          INDENT "return" Expr [";"] DEDENT

LocalparamDecl ::= "localparam" IDENT "=" Expr
                { "," IDENT "=" Expr } ";"

AssignStmt ::= ["assign"] Lvalue "=" Expr ";"
           | Lvalue "<=" Expr ";"
           | ["assign"] Lvalue CompoundOp Expr ";"

CompoundOp ::= "+=" | "-=" | "*=" | "/=" | "%="
           | "&=" | "|=" | "^=" | "<<=" | ">>="
           | "<<<=" | ">>>="

EventExpr  ::= Expr
SeqBlock   ::= "seq" "(" [ ("pos" | "neg") ":" ] EventExpr
               [ "," ("pos" | "neg") ":" EventExpr ] ")" BlockStmt
CombBlock  ::= "comb" BlockStmt
InitialBlock ::= "initial" BlockStmt

IfStmt ::= "if" "(" Expr ")" BlockStmt
          [ "else" BlockStmt ]

ForStmt ::= "for" "(" IDENT "=" Expr ";" Expr ";"
                ForStep ")" BlockStmt
ForStep ::= IDENT "=" Expr
         | IDENT CompoundOp Expr
         | IDENT "++"

CaseStmt ::= ("case" | "casez" | "casex") "(" Expr ")"
```

```
           "{" { CaseItem } "}"
CaseItem ::= PatternList ":" Statement
          | "default" ":" Statement
PatternList ::= Expr { "," Expr }

GenForStmt ::= "`for" "(" TICK_IDENT "=" Expr ";" Expr ";"
                       TICK_IDENT "=" Expr ")" BlockStmt
GenIfStmt ::= "`if" "(" Expr ")" BlockStmt
                      [ "`else" BlockStmt ]

InstanceStmt ::= IDENT
               [ "#(" NamedParam { "," NamedParam } ")" ]
               IDENT "{" { Connection } "}" [";"]

NamedParam ::= "." IDENT "(" Expr ")"
NamedConnection ::= "." IDENT [ "(" Expr ")" ]
Connection ::= NamedConnection
            | STRING "=>" STRING

BlockStmt ::= "{" { Statement } "}"
SelectSuffix ::= "[" Expr "]" | "[" Expr ":" Expr "]"
              | "[" Expr ("+:" | "-:") Expr "]" | "." IDENT
Lvalue ::= IDENT { SelectSuffix }
        | "(" Lvalue ")" { SelectSuffix }
        | "{" Lvalue { "," Lvalue } "}"
```

Notes on the grammar:

- `TICK_IDENT` denotes a backtick-prefixed identifier (e.g., `` `i ``).
- Template identifiers (e.g., `data_`i`) are formed at the lexical level by merging adjacent `IDENT`, `TICK`, and `TICK_IDENT` tokens.
- Expressions are parsed by a Pratt parser and support all operators listed in the operator precedence table.
- The grammar summarizes syntax; context rules restrict declarations, returns, assertions and processes to their permitted scopes. Enum bases are logic/bit/int; packed struct fields have no port directions or unpacked dimensions. Function arguments are inputs. Bundle fields require input/output directions.
- Native interface ports require a modport and inherit parameters; internal interface instances may set parameters and constructor connections. Instance and port arrays are not supported.
- `INDENT`/`DEDENT` summarize the indentation-based defun body. RTL `function` bodies use braces.
- Special expression forms include concatenation `{a, b}`, replication `{n{expr}}`, width or named-type casts, field access, indexed part selects, and set membership `x inside {v1, v2, ...}`.
- Compound assignment operators are desugared at parse time into regular assignments.